Ain Shams University

Faculty of Engineering

Senior2 Mechatronics Eng. Department

Spring 2026

CSE480: Machine Vision

Project Milestone 1

Name	ID	Section
Hla Ehab Fawzy	2100708	1
Arwa Ramadan Abdel Fattah	2100647	1

Submitted to:

Eng. Mohamed Mohanna

Eng. Sarah

Table of Contents

1.0 Introduction 6
2.0 System Architecture 6
3.0 Math and Algorithms 7
3.1. Image I/O & Color Conversion 7
3.1.1 Image Reading and Writing 7
3.1.2 RGB to Grayscale Conversion 7
3.1.3 Grayscale to RGB Expansion 8
3.2. Core Foundation Operations 8
3.2.1 Image Normalization 8
Min-Max Normalization 8
Z-Score Normalization 9
Mean Normalization 9
Implementation Strategy 9
3.2.2 Pixel Clipping 9
Zero Padding (Constant) 10
Replicate Padding (Edge) 10
Reflect Padding 10
Implementation Strategy: 10
3.2.4 2D Convolution 10
3.2.5 Spatial Filter 11
Implementation Strategy 11
3.3. Image Processing Techniques 11
3.3.1 Mean (Box) Filter 11
3.3.2 Gaussian Filter 12
2D Gaussian Function 12
Kernel Generation Algorithm 12
Filtering Pipeline 12
3.3.3 Median Filter 13
Algorithm 13
Implementation Considerations 13
Properties 13

3.3.4 Thresholding Techniques 13
Global Thresholding 13
Otsu's Thresholding 14
Adaptive Thresholding 14
Implementation Notes 14
3.3.5 Sobel Gradients 15
Sobel Kernels 15
Algorithm 15
3.3.6 Bit-Plane Slicing 16
Algorithm 16
3.3.7 Histogram & Histogram Equalization 16
Image Histogram 16
Histogram Equalization 17
Algorithm 17
Properties 17
The Algorithm 17
3.4. Geometric Transformations 18
3.4.1 Image Resizing 18
3.4.2 Image Rotation 19
3.4.3 Image Translation 20
3.5. Feature Extraction 21
3.5.1 Global Descriptors 21
3.5.1.1 Color Histogram 21
3.5.1.2 Basic Statistical Features 22
3.5.2 Gradient-Based Descriptors 23
3.5.2.1 Histogram of Oriented Gradients (HOG) 23
3.5.2.2 Edge Histogram Descriptor (EHD) 24
3.6. Drawing Primitives 25
3.6.1 Point Drawing 25
3.6.2 Line Drawing (Bresenham's Algorithm) 25
The Core Logic: Error Accumulation 26
Algorithm 26

General Case 26
3.6.3 Rectangle Drawing 27
Filled Mode 27
Outline Mode 27
3.6.4 Polygon Drawing 27
Boundary Clipping 27
3.7. Text Rendering 27
Bitmap Font Architecture 27
Scaling and Rendering Logic 28
3.8. Additional Image Processing Techniques 28
3.8.1 Laplacian Edge Detection 28
Mathematical Definition 28
Discrete Approximation 28
Properties 29
3.8.2 Morphological Operations 29
Erosion 29
Dilation 29
Opening and Closing 29
Structuring Elements 29
4.0 Results 29
5.0 Conclusion 42
6.0 Repository Link 42

Table of Figures

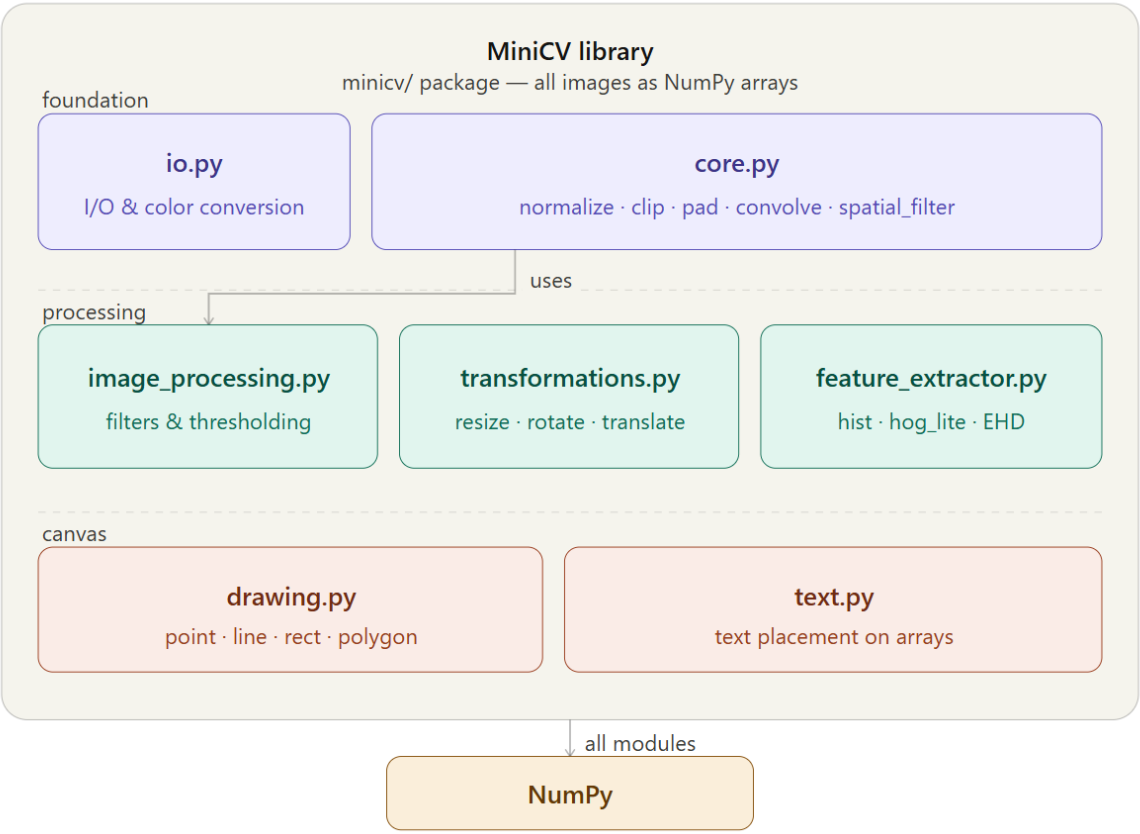
Figure 4.0-1: Grayscale Image 30
Figure 4.0-2: RGB Image 30
Figure 4.0-3: RGB to Grayscale Conversion 31
Figure 4.0-4: Median Filter 31
Figure 4.0-5: Mean Filter 32
Figure 4.0-6: Gaussian Filter 32
Figure 4.0-7: Gamma Corrected Image 33
Figure 4.0-8: Histogram Equalization 33
Figure 4.0-9: Laplacian Filter 34
Figure 4.0-10: Sobel Filter Magnitude 34
Figure 4.0-11: Sobel Filter Direction 35
Figure 4.0-12: Global Thresholding 36
Figure 4.0-13: Ostu's Thresholding 36
Figure 4.0-14: Adaptive Thresholding 37
Figure 4.0-15: Bit Slicing 37
Figure 4.0-16: Adding Text on Grayscale Image 38
Figure 4.0-17: Adding Text on RGB Image 38
Figure 4.0-18: Adding Drawing on Grayscale Image 39
Figure 4.0-19: Adding Drawing on RGB Image 39
Figure 4.0-20 Normalize(min_max, z_score, mean_norm)- clipping(clipped image,original &clipped histogram)-Padding(constant, reflect, edge) 40
Figure 4.0-21Color Histogram, HOG Lite, Edge Histogram Descriptor 41
Figure 4.0-22 Resize (nearest-Bilinear)_Rotate(nearest-Bilinear)_Translate 42

1.0 Introduction

The MiniCV library is a from-scratch implementation of essential computer vision algorithms, designed to emulate a subset of OpenCV functionality using only NumPy, Pandas, Matplotlib, and Python's standard library. This report documents the mathematical foundations and algorithmic approaches for each component of the library.

The library encompasses nine major functional areas: image I/O and color conversion, core foundation operations (normalization, padding, convolution), image processing techniques (filtering, thresholding, gradient detection), geometric transformations, feature extraction, drawing primitives, and text rendering. Each component is implemented with careful attention to mathematical correctness, computational efficiency, and software engineering best practices.

2.0 System Architecture



3.0 Math and Algorithms

3.1. Image I/O & Color Conversion

3.1.1 Image Reading and Writing

The IO module provides a robust interface for data acquisition and persistence, ensuring that images are correctly formatted for the computer vision pipeline.

Mathematical Representation:

For an RGB image I, the data is stored as a 3D tensor where each spatial location (i, j) contains a color vector:

I[i, j] = [R(i,j), G(i,j), B(i,j)] where $0 \leq i < H$, $0 \leq j < W$

Data Normalization and Quantization:

The implementation handles two primary data types to balance memory efficiency and computational precision:

Normalization: For arithmetic stability, images are mapped from the discrete uint8 space to a continuous float32 space:

$$f = \frac{u}{225.0}$$

Denormalization (Exporting): To save images, the data must be clipped to the valid range [0, 1] and quantized back to uint8:

$$u = clip(round(f \times 255), 0{,}255)$$

Algorithm Implementation:

Strict Validation: The read_image function performs type-checking and path verification to ensure the file system integrity.

Adaptive Loading: Uses mpimg.imread which automatically identifies the file header (JPEG vs. PNG).

3.1.2 RGB to Grayscale Conversion

Grayscale conversion uses the luminosity method, which weights RGB channels according to human perception. The ITU-R BT.601 standard provides perceptually calibrated coefficients:

Formula:

$$Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

These coefficients prioritize the green channel (58.7%), reflecting the human eye's higher sensitivity to middle-wavelength light.

Implementation Considerations:

- Alpha Channel Slicing: The algorithm detects 4-channel inputs (RGBA) and automatically slices the array ([:, :, :3]) to ignore transparency data before conversion.
- Vectorization: Instead of pixel-wise loops, the conversion utilizes NumPy's internal broadcasting for high-speed matrix multiplication.
- Rounding Correction: To maintain maximum visual fidelity, the resulting float32 values are rounded using np.round before being cast back to $uint8$, preventing the "darkening" effect of standard integer truncation.

3.1.3 Grayscale to RGB Expansion

For visualization and multi-modal compatibility, the library supports the expansion of 1-channel grayscale maps into a 3-channel RGB format.

Mathematical Logic:

This process creates a "pseudo-color" image where all three-color channels are assigned the same intensity value, resulting in a neutral gray appearance:

$$R(i, j) = G(i, j) = B(i, j) = Y(i, j)$$

Algorithm:

Axis Stacking: The implementation uses np.stack to duplicate the 2D array three times along a new depth axis (Axis 2).

3.2. Core Foundation Operations

3.2.1 Image Normalization

Normalization is used to rescale pixel intensity values into a standard range, which is essential for improving numerical stability and ensuring consistent behavior across different image processing and machine learning algorithms.

The implemented function normalize() supports multiple normalization strategies:

Min-Max Normalization

This method rescales pixel values to the range [0, 1]:

$$I_{norm} = \frac{I - I_{min}}{I_{max} - I_{min}}$$

- Preserves relative intensity differences
- If the image is constant (min = max), the output is a zero array

Z-Score Normalization

Standardizes pixel values to have zero mean and unit variance:

$$I_{norm} = \frac{I - \mu}{\sigma}$$

Where:

- $\mu$is the mean intensity
- $\sigma$is the standard deviation
- Output may contain negative values
- Useful in machine learning pipelines
- If standard deviation is zero → returns zeros

Mean Normalization

Centers values around zero and scales by intensity range:

$$I_{norm} = \frac{I - \mu}{I_{max} - I_{min}}$$

- Produces values approximately in range [-0.5, 0.5]
- More stable than min-max in some cases

Implementation Strategy

- Input is converted to float32
- Works for both grayscale (H, W) and RGB (H, W, 3)
- For RGB images:
    - Normalization is applied globally across all channels
    - This preserves inter-channel relationships

3.2.2 Pixel Clipping

Pixel clipping ensures that all pixel values lie within a valid intensity range, preventing overflow and underflow.

Mathematical Definition:

$$clip(I, low, high) = \begin{cases} low & I < low \\ I & low \leq I \leq high \\ high & I > high \end{cases}$$

Implementation Strategy:

- Validate input range such that:

$$low < high$$

- Apply element-wise clipping using a vectorized operation:
  - Compare each pixel against bounds
  - Values below low are replaced with low
  - Values above high are replaced with high
  - Preserve original image shape and data type

3.2.3 Image Padding

Padding extends image boundaries to handle border pixels during convolution. Three essential modes:

Zero Padding (Constant)

Adds a constant border:

$$I_{padded} = pad(I, p, c)$$

Effect: introduces artificial edges.

Replicate Padding (Edge)

Repeats border values outward:

$$I[-p:0] = I[0], I[W:W+p] = I[W-1]$$

Effect: preserves intensity continuity at edges

Reflect Padding

Mirrors the image at boundaries:

$$I_{padded}(j) = \begin{cases} I(|\,j\,|) & j < 0 \\ I(2W - j - 1) & j \geq W \end{cases}$$

Effect: preserves gradients

Implementation Strategy:

- Uses np.pad() internally
- Automatically handles:
  - Grayscale → (H, W)
  - RGB → (H, W, 3) (padding only spatial dimensions)
- Supports multiple pad formats:
  - single int
  - tuple

3.2.4 2D Convolution

The module implements a low-level 2D convolution engine designed for grayscale processing, adhering to the mathematical definition of convolution while ensuring robust input handling.

Mathematical Definition

For image I and kernel K of size (2m+1) × (2n+1):

$(I * K)[i, j] = \Sigma\Sigma\ I[i+u, j+v] \times K[u, v]$ where $u \in [-m, m]$, $v \in [-n, n]$

Note: This is actually correlation. True convolution flips the kernel: K[-u, -v]. For symmetric kernels, they're equivalent.

Implementation Strategy

- Validate kernel: must be 2D, non-empty, odd dimensions, numeric type
- Pad input image based on kernel size: pad_width = (kernel_height // 2, kernel_width // 2)
- Sliding Window: The algorithm traverses the image coordinates (i, j) to extract a local neighborhood (window) from the padded source.
- Vectorized Reduction: For each window, the library utilizes NumPy broadcasting to perform element-wise multiplication with the kernel, followed by a global summation:

Output [i,j] = Sum (Window * Kernel)

3.2.5 Spatial Filter

The module implements a high-level 2D spatial filter for both grayscale and RGB images. For RGB images, two different methods are used: per-channel or HSI.

Implementation Strategy

- **Dimensional Routing:** Analyzes the input rank to automatically distinguish between Grayscale (Rank 2) and RGB (Rank 3) processing pipelines.
- **Grayscale Execution:** Directly passes 2D arrays to the convolve2d engine for immediate processing.
- **Per-Channel Strategy:** Decomposes RGB images into three separate planes, applies the kernel to each independently, and recomposes them using np.stack.
- **HSI Intensity Processing:** Converts RGB data to the HSI color space. Filters only the Intensity (I) channel to modify brightness while preserving original chromaticity (Hue/Saturation).
- Reverts the result back to RGB.

3.3. Image Processing Techniques

3.3.1 Mean (Box) Filter

The mean filter replaces each pixel with the average of its neighbourhood, providing simple noise reduction through local averaging.

Kernel Definition:

For kernel size k × k:

$K[i, j] = 1/(k^2)$ for all i, j

Example 3×3 kernel: [[1/9, 1/9, 1/9], [1/9, 1/9, 1/9], [1/9, 1/9, 1/9]]

Properties and Applications

- Linear filter (sum of weighted inputs)
- Reduces high-frequency noise but blurs edges
- Separable: 2D convolution = 1D row convolution + 1D column convolution (computational optimization)
- Larger kernels = more blur, more noise reduction

3.3.2 Gaussian Filter

The Gaussian filter provides weighted averaging where nearby pixels have more influence than distant ones, following a Gaussian distribution. This produces superior smoothing compared to box filtering.

2D Gaussian Function

The Gaussian kernel is generated from the 2D Gaussian function:

$G(x, y) = (1 / (2\pi\sigma^2)) \times \exp(-(x^2 + y^2) / (2\sigma^2))$

where:

- σ (sigma) = standard deviation, controls spread
- x, y = distances from kernel center
- Larger σ = wider spread = more blur

Kernel Generation Algorithm

The gaussian_kernel function utilizes the separability property of the Gaussian function G(x,y) = G(x) * G(y) for efficient construction:

- **Coordinate Grid:** A 1D linear space is generated, centered at zero, spanning the specified kernel size.
- **Outer Product:** Instead of calculating the full 2D formula for every cell, a 1D Gaussian vector is computed and used to generate the 2D matrix via np.outer(gauss, gauss).
- **Normalization:** The resulting matrix is divided by kernel.sum(). This ensures the energy is preserved (sum = 1), preventing the image from brightening or darkening during filtering.

Filtering Pipeline

The high-level gaussian_filter acts as an abstraction layer:

- **Parameter Coupling:** It calculates the required kernel based on the user's size and sigma inputs.
- **Convolution Integration:** It passes the generated kernel to the Core.spatial_filter module, which handles the complex tasks of padding and multi-channel (RGB) processing.

Properties and Applications

- Separable: G(x,y) = G(x) × G(y), enabling faster computation
- Isotropic: rotation-invariant smoothing
- No ringing artifacts (unlike ideal low-pass filter)
- Foundation for many algorithms: edge detection, feature detection, scale-space analysis

3.3.3 Median Filter

The median filter replaces each pixel with the median value of its neighbourhood. Unlike linear filters, it's a non-linear operation particularly effective against salt-and-pepper noise.

Algorithm

For each pixel (i, j) and kernel size k:

1. Extract neighborhood: N = I[i-k//2 : i+k//2+1, j-k//2 : j+k//2+1]
2. Flatten neighborhood to 1D array
3. Sort values: N_sorted = sort(N)
4. Return middle value: median = N_sorted[len(N)//2]

Implementation Considerations

Median filtering requires explicit loops because:

- Median is non-linear (cannot be expressed as convolution)
- Requires sorting at each pixel position
- No vectorized NumPy equivalent for 2D sliding median

Properties

- Edge-preserving: sharp boundaries remain intact
- Removes impulse noise (salt and pepper) effectively (isolated extreme values)
- Does not reduce Gaussian noise as well as averaging
- Output pixels are always actual values from input (no interpolation)

3.3.4 Thresholding Techniques

Thresholding converts grayscale images to binary by classifying pixels as foreground or background based on intensity values.

Global Thresholding

Applies a single threshold T to the entire image:

I_binary[i, j] = { 255 if I[i, j] > T; 0 otherwise }

Simple but fails when illumination varies spatially. Threshold selection is critical.

Otsu's Thresholding

It assumes the image contains two classes of pixels (bimodal distribution). It searches for the threshold that minimizes the intra-class variance.

Algorithm:

1. Compute the Histogram: Count how many pixels exist at every intensity level from 0 to 255.
2. Iterate Through All Possible Thresholds: The algorithm tests every possible value of T from 0 to 255.
3. Calculate Class Weights and Variances: For each test threshold :
   - Separate pixels into Class 1 (Background) and Class 2 (Foreground).
   - Calculate the weight , mean , and variance for both classes.
   - For each threshold candidate $t \in [0, 255]$:

     a. Compute class probabilities:

     $\omega_0(t) = \Sigma\, p(i)$ for $i \in [0, t]$ (background)

     $\omega_1(t) = \Sigma\, p(i)$ for $i \in [t+1, 255]$ (foreground)

     b. Compute class means:

     $\mu_0(t) = (\Sigma\, i \times p(i)$ for $i \leq t) / \omega_0(t)$

     $\mu_1(t) = (\Sigma\, i \times p(i)$ for $i > t) / \omega_1(t)$

     c. Compute between-class variance:

     $\sigma^2_between(t) = \omega_0(t) \times \omega_1(t) \times (\mu_0(t) - \mu_1(t))^2$

4. Find the Minimum: The optimal threshold T^ is the one where the weighted sum of variances is the lowest

Intuition:

Optimal threshold maximally separates background and foreground intensity distributions. Best for bimodal histograms (two distinct peaks).

Adaptive Thresholding

Computes local threshold for each pixel based on its neighbourhood, handling variable illumination.

Mean Adaptive:

T[i, j] = mean(N[i, j]) - C

where N[i,j] is the k×k neighborhood and C is a constant offset.

Implementation Notes

- Adaptive thresholding can be implemented via local filtering (convolution with mean kernel) followed by subtraction

- Block size k and constant C are hyperparameters (typical: k=11-31, C=2-10)
- Larger blocks capture broader illumination trends but may miss fine details

3.3.5 Sobel Gradients

Sobel operators detect edges by approximating image gradients using discrete differentiation kernels with Gaussian smoothing.

Sobel Kernels

Horizontal (x-direction) gradient:

G_x = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]

Vertical (y-direction) gradient:

G_y = [[-1, -2, -1], [0, 0, 0], [1, 2, 1]]

Algorithm

The sobel_gradients function follows a structured arithmetic flow:

1. **Spatial Filtering:** The image is convolved with K_x and K_y via the Core.spatial_filter module to produce partial derivatives G_x and G_y.

2. **Magnitude Approximation:** To maximize computational efficiency, the edge strength (magnitude) is calculated using the absolute sum approximation rather than the Euclidean distance:

$$G = |G_x| + |G_y|$$

3. **Direction:** To get the direction of the edges, we use:

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

4. **Data Normalization:** The resulting magnitude is passed through Core.clip_pixels to ensure values remain within the valid [0, 255] range, then cast back to uint8 for visualization.

5. **Orientation Map (HSV):** To visualize the vector flow, the implementation maps the gradient direction to the **HSV color space**:

   Hue: Encodes the angle θ (mapped to 0-180 for symmetry).

   Saturation: Fixed at maximum (255) for vividness.

   Value: Encoded by the gradient magnitude, ensuring that only "strong" edges appear brightly colored, while uniform regions remain dark.

Properties and Applications

- Edge detection: threshold gradient magnitude
- Feature extraction: HOG descriptors use gradient histograms
- Image sharpening: add scaled gradient to original
- Foundation for Canny edge detector

3.3.6 Bit-Plane Slicing

Bit-plane slicing decomposes a grayscale image into binary images representing each bit position, revealing the contribution of each bit to the overall appearance.

Algorithm

Knowing that an 8-bit grayscale pixel can be represented as:

$I[i, j] = b_7 \times 2^7 + b_6 \times 2^6 + \ldots + b_1 \times 2^1 + b_0 \times 2^0$

where $b_k \in \{0, 1\}$ is the k-th bit (b_7 is MSB, b_0 is LSB).

To extract bit plane k:

bit_plane_k = (I >> k) & 1

This:

- Right-shifts pixel values by k positions
- Applies bitwise AND with 1 to isolate least significant bit

Result is binary image (0 or 1 values), typically scaled to {0, 255} for visualization.

Properties and Applications

- MSB (bit 7): Contains most significant visual information, resembles thresholded image
- Middle bits (4-6): Capture shape and structure details
- LSBs (0-3): Appear noise-like, contain subtle variations

  Bit slicing is used in:
- Image compression: discard lower bit planes with minimal perceptual loss
- Steganography: hide data in LSBs (least detectable)
- Understanding data contribution: analyse which bits carry visual information

3.3.7 Histogram & Histogram Equalization

Image Histogram

The histogram represents the frequency distribution of pixel intensities:

$H[k] = \#\{\text{pixels with intensity } k\} \quad \text{for } k \in [0, 255]$

Computed efficiently with NumPy:

```
H = np.histogram(image, bins=256, range=(0, 255))[0]
```

Histogram Equalization

Histogram equalization enhances contrast by spreading out intensity values to approximate a uniform distribution.

Mathematical Foundation:

The transformation is based on the cumulative distribution function (CDF):

$CDF[k] = \Sigma H[i]$ for $i \in [0, k]$

Algorithm

1. **Intensity Frequency Distribution:** The image is flattened into a 1D array to compute a 256-bin histogram, capturing the count of every gray level from 0 to L-1 (L=256).
2. **Cumulative Distribution Function (CDF):** Using np.cumsum, the algorithm calculates the cumulative sum of the histogram. When divided by the total number of pixels, this yields the **normalized CDF** (s), representing the probability P_r(r).
3. **Transfer Function Application:** To map the original intensities to the equalized range, the implementation applies the transformation formula:

$$z = L * s - 1$$

This maps the cumulative probabilities back to the [0, 255] integer space.

4. **Intensity Mapping:** The original image acts as an index into the look-up table (z). This replaces every pixel with its new "equalized" value, effectively flattening the histogram.

Properties

- Increases global contrast, especially in low-contrast images
- May amplify noise in near-uniform regions
- Ideally produces uniform histogram (in practice, discrete nature prevents perfect uniformity)

3.3.8 Gamma Correction (Power-Law Transformation)

Gamma correction is a non-linear operation used to adjust the luminance or tristimulus values in video or still image systems. It is particularly effective for correcting images that are either underexposed (too dark) or overexposed (too bright) by remapping the intensity levels according to a power-law relationship.

The Algorithm

- **Normalization & Precision:** The input image is converted to float32 to prevent rounding errors during exponentiation.
- **Dynamic Range Extraction:** The algorithm identifies the current intensity bounds (r_min and r_max). This ensures the transformation is relative to the actual content of the image rather than a theoretical range.

- **Power-Law Application:** The core of the function implements the normalized power-law equation:

$$S = Smin + (\frac{r^{\gamma} - r^{\gamma}min}{r^{\gamma}max - r^{\gamma}min})(Smax - Smin)$$

- **Non-Linear Response:**

gamma < 1 (Encoding/Brightening): Compresses high-intensity values and expands low-intensity values, making dark regions more visible.

gamma > 1 (Decoding/Darkening): Compresses low-intensity values and expands high-intensity values, increasing contrast in bright regions.

3.4. Geometric Transformations

Geometric transformations modify the spatial arrangement of image pixels.
All implemented operations use inverse mapping, where each output pixel is mapped back to a corresponding source location to avoid holes in the output.

3.4.1 Image Resizing

Resizing changes the spatial resolution of an image by resampling pixel values from the source grid to a new grid.

Mathematical Definition:

Let the original image be of size $H \times W$, and the target size be $H' \times W'$.

Scaling factors:

$$s_y = \frac{H}{H'}, s_x = \frac{W}{W'}$$

Each destination pixel (i', j')maps to source coordinates:

$$y = i' \cdot s_y, x = j' \cdot s_x$$

Nearest Neighbor Interpolation:

$$i = round(y), j = round(x)$$
$$I_{out}[i', j'] = I_{in}[i, j]$$

Bilinear Interpolation:

$$x_0 = \lfloor x \rfloor, x_1 = x_0 + 1$$
$$y_0 = \lfloor y \rfloor, y_1 = y_0 + 1$$

$$dx = x - x_0, dy = y - y_0$$
$$I_{out} = (1 - dx)(1 - dy)I[y_0, x_0] + dx(1 - dy)I[y_0, x_1] + (1 - dx)dyI[y_1, x_0] + dxdyI[y_1, x_1]$$

Implementation Strategy:

- Generate destination grid using:
  - np.indices(new_shape)
- Apply inverse mapping:
  - Compute floating source coordinates
- Handle two interpolation modes:

Nearest:

- Round coordinates
- Clip to image bounds
- Direct indexing

Bilinear:

- Compute 4 neighboring pixels
- Compute fractional distances (dx, dy)
- Perform weighted interpolation
- Support RGB via broadcasting
- Output retains same dtype as input

3.4.2 Image Rotation

Rotation transforms image coordinates around the image center by an angle θ.

Mathematical Definition:

Rotation is defined using:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Inverse Mapping (Used in Implementation):

Instead of forward rotation, inverse rotation is used:

$$x_{src} = (x - c_x)\cos\ \theta + (y - c_y)\sin\ \theta + c_x$$
$$y_{src} = -(x - c_x)\sin\ \theta + (y - c_y)\cos\ \theta + c_y$$

Where:

- $(c_x, c_y)$is the image center

Implementation Strategy:

- Convert angle from degrees to radians
- Compute:
  - $\cos(\theta), \sin(\theta)$
- Generate full destination grid using np.indices
- Shift coordinates to image center:
  - $x - c_x, y - c_y$
- Apply inverse rotation
- Shift back to original coordinate system

Pixel Sampling:

Two interpolation methods are supported:

Nearest Neighbor:

- Round source coordinates
- Assign nearest pixel value

Bilinear Interpolation:

- Compute surrounding pixels
- Perform weighted interpolation

Boundary Handling:

- A validity mask is used:
  - Only pixels inside source bounds are assigned
- Pixels outside are set to:

$$0(\text{black})$$

3.4.3 Image Translation

Translation shifts the image spatially without changing orientation or scale

Mathematical Definition:

$$x' = x + t_x, y' = y + t_y$$

Implementation via Inverse Mapping:

$$x_{src} = x' - t_x, y_{src} = y' - t_y$$

Implementation Strategy:

Initialize output image with zeros

- Compute source and destination slicing boundaries:

source region → destination region

Key Observations:

- No interpolation required (integer shift)
- Efficient implementation using slicing
- Handles both positive and negative shifts

3.5. Feature Extraction

Feature descriptors convert images into numerical representations suitable for comparison, classification, and retrieval. Two categories are required: global descriptors (whole-image features) and gradient-based descriptors (local texture/shape features).

3.5.1 Global Descriptors

3.5.1.1 Color Histogram

Represents the distribution of pixel intensities (or colors) in the image.

Mathematical Definition:

For grayscale image:

$$H[k] = \#\{I(x, y) \in \text{bin}_k\}$$

For RGB image:

$$\text{Feature} = [H_R, H_G, H_B]$$

Normalization:

$$H_{norm} = \frac{H}{\sum H + \epsilon}$$

Implementation Strategy:

- Convert image to float
- Normalize range:
  - If image in [0,1] → scale to [0,255]
- Compute histogram using:
  - fixed range $[0,256)$

Grayscale Case:

- Compute single histogram

RGB Case:

- Compute histogram for each channel separately
- Concatenate:

$$[H_R \mid H_G \mid H_B]$$

- Apply L1 normalization (sum = 1)

3.5.1.2 Basic Statistical Features

Mathematical Definition

Let pixel values be x:

Mean:

$$\mu = \frac{1}{N}\sum x$$

Standard Deviation:

$$\sigma = \sqrt{\frac{1}{N}\sum(x-\mu)^2}$$

Skewness:

$$\text{Skewness} = \frac{E[(x-\mu)^3]}{\sigma^3 + \epsilon}$$

Kurtosis:

$$\text{Kurtosis} = \frac{E[(x-\mu)^4]}{\sigma^4 + \epsilon}$$

Implementation Strategy:

- Convert image → grayscale:

$$I = 0.299R + 0.587G + 0.114B$$

- Normalize values to [0,1]
- Flatten image into 1D vector
- Compute:
  - mean
  - std
  - skewness
  - kurtosis
- Return 4D feature vector

3.5.2 Gradient-Based Descriptors

3.5.2.1 Histogram of Oriented Gradients (HOG)

HOG captures local texture and shape by computing gradient orientation histograms over dense grid cells.

Mathematical Definition

Gradient Computation

$$G_x = I(x+1, y) - I(x-1, y)$$
$$G_y = I(x, y+1) - I(x, y-1)$$

Magnitude

$$M = \sqrt{G_x^2 + G_y^2}$$

Orientation

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

Mapped to:

$$\theta \in [0°, 180°)$$

Histogram

$$H[k] = \sum M \text{for orientations in bin } k$$

Normalization

$$H_{norm} = \frac{H}{\sum H + \epsilon}$$

Implementation Strategy

- Convert image → grayscale float
- Compute gradients using central difference:
  - no convolution used
  - fully vectorized slicing
- Compute:
  - magnitude
  - orientation
- Build histogram:
  - bins = n_bins (default 9)
  - weighted by magnitude
- Normalize histogram (L1)

3.5.2.2 Edge Histogram Descriptor (EHD)

Mathematical Definition

Same gradient definitions as HOG:

$$G_x, G_y, M, \theta$$

Spatial Division

Divide image into grid:

$$grid = (R, C)$$

Each cell:

$$Cell_{i,j}$$

Per-Cell Histogram

$$H_{cell}[k] = \sum M \text{for orientations in bin } k$$

Normalization (per cell)

$$H_{cell} = \frac{H_{cell}}{\sum H_{cell} + \epsilon}$$

Final Descriptor

$$Feature = [H_1 \ | \ H_2 \ | \ \dots \ | \ H_{R \times C}]$$

Implementation Strategy

- Convert image → grayscale
- Compute gradients using central difference
- Compute:
  - magnitude
  - orientation
- Divide image into grid using:
  - np.array_split
- For each cell:
  - extract region
  - compute histogram (weighted by magnitude)
  - normalize (L1)

3.6. Drawing Primitives

Drawing operations modify NumPy arrays directly, enabling annotation and visualization without external libraries.

3.6.1 Point Drawing

Setting a single pixel value:

I[y, x] = color

For thickness > 1, draw a filled square centered at (x, y):

r = thickness // 2

I[y-r:y+r+1, x-r:x+r+1] = color

With boundary clipping to prevent out-of-bounds access.

3.6.2 Line Drawing (Bresenham's Algorithm)

Bresenham’s algorithm is an efficient method for determining which pixels in a 2D integer grid should be selected to form a close approximation of a straight line between two points.

The Core Logic: Error Accumulation

Unlike simpler algorithms that use floating-point division and rounding (which are computationally expensive), Bresenham's uses integer arithmetic only.

- **Incremental Steps:** The algorithm always moves one step in the primary direction (the axis with the largest change).
- **Decision Variable:** It maintains an "error" variable that tracks the vertical/horizontal distance between the current pixel and the ideal mathematical line.
- **Pixel Selection:** When the error exceeds a specific threshold, the algorithm increments the coordinate on the secondary axis and resets the error term.

Implementation Details

- **Octant Handling:** The algorithm calculates the absolute differences (Delta x, Delta y) and the step direction (positive or negative) to handle lines in any direction.
- **Loop Justification:** A single while loop iterates from the starting point to the destination. Inside this loop, the draw_point logic is invoked to modify the canvas.

Algorithm

For line from (x_0, y_0) to (x_1, y_1):

$dx = x_1 - x_0$, $dy = y_1 - y_0$

$D = 2 \times dy - dx$ (decision variable)

$y = y_0$

For x from x_0 to x_1:

```
  plot(x, y)
  if D > 0:
    y = y + 1
    D = D - 2×dx
  D = D + 2×dy
```

General Case

Handle all octants by:

- Swapping coordinates for steep lines (|dy| > |dx|)
- Adjusting increment direction for negative slopes
- Ensuring start point is to the left of end point

3.6.3 Rectangle Drawing

Filled Mode

Leverages the power of NumPy slicing. By identifying the bounding box [y_{start}:y_{end}, x_{start}:x_{end}], the algorithm performs a bulk assignment of the color value. This is extremely efficient as it bypasses pixel-by-pixel loops in Python.

Outline Mode

Deconstructs the rectangle into four discrete boundary segments. It calls draw_line for the Top, Bottom, Left, and Right edges, ensuring that the thickness property is applied consistently across all borders.

3.6.4 Polygon Drawing

Polygons are treated as a closed series of connected vectors:

- **Vertex Chaining:** The draw_polygon function iterates through a list of vertices. It connects vertex i to vertex i+1, using the modulo operator (i + 1) % num_vertices to automatically connect the final vertex back to the first, closing the shape.
- **Coordinate Offsetting:** Supports a center parameter, allowing shapes to be defined by local relative coordinates and then translated to a specific global position (cx, cy) on the canvas.
- **Regular Polygon Generation:** A specialized helper uses polar-to-cartesian conversion. It calculates vertex positions at equal angular intervals $2\pi/n$ along a circle of a given radius, then passes these vertices to the general polygon engine.

Boundary Clipping

All drawing functions must clip coordinates:

x_clipped = clip(x, 0, W-1)

y_clipped = clip(y, 0, H-1)

This prevents array index errors when drawing near/outside image boundaries.

3.7. Text Rendering

The Text class provides a custom implementation for overlaying alphanumeric characters onto NumPy arrays. Unlike standard libraries that rely on external TrueType font files, this module uses a self-contained **bitmap font** system for maximum portability and zero dependencies.

Bitmap Font Architecture

The core of the rendering engine is a hardcoded dictionary (_FONT) that defines characters using a 5x7 binary matrix:

- **Data Structure:** Each character is represented by a nested list where 1 denotes a "filled" pixel and 0 represents empty space.
- **Character Set:** Supports uppercase letters (A-Z), numerical digits (0-9), and basic punctuation (periods and hyphens).
- **Normalization:** The put_text method automatically converts input strings to uppercase to match the internal dictionary keys, defaulting to a "space" matrix for unrecognized characters.

Scaling and Rendering Logic

The rendering process simulates a "virtual cursor" that moves across the canvas:

- **Font Scaling:** Scaling is achieved by treating each "1" in the bitmap as a block of pixels rather than a single point. If font_scale=2, every active bit in the 5x7 matrix is rendered as a 2 *2 square on the canvas.
- **Nested Iteration:** The algorithm uses a four-level loop structure:
- **Character Loop:** Iterates through the input string.
- **Bitmap Traversal:** Scans the 5x7 matrix for active bits.
- **Scaling Blocks**: Draws the square representing the scaled pixel.
- **Boundary Safety:** Checks every coordinate against the canvas dimensions (H, W) before assignment to prevent memory errors.
- **Kerning and Spacing:** After each character is rendered, the current_x position is incremented by (cols + 1) * font_scale. This ensures consistent proportional spacing between letters.

3.8. Additional Image Processing Techniques

Two additional techniques are required beyond the core set. Recommended options:

3.8.1 Laplacian Edge Detection

The Laplacian operator detects edges by computing the second derivative of the image, highlighting regions of rapid intensity change.

Mathematical Definition

The Laplacian is the sum of second partial derivatives:

$\nabla^2 I = \partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$

Discrete Approximation

Standard 3×3 Laplacian kernel:

K = [[0, 1, 0], [1, -4, 1], [0, 1, 0]]

Alternative (includes diagonals):

K = [[1, 1, 1], [1, -8, 1], [1, 1, 1]]

Properties

- Isotropic (rotation-invariant)
- Detects edges as zero-crossings
- Very sensitive to noise (often combined with Gaussian smoothing → LoG)
- Negative values indicate dark-to-light transitions, positive for light-to-dark

3.8.2 Morphological Operations

Morphological operations process binary images based on shape using structuring elements.

Erosion

Shrinks objects by removing boundary pixels:

$(I \ominus S)[i, j] = \min\{I[i+u, j+v] : (u,v) \in S\}$

Output pixel is foreground (1) only if all pixels under structuring element S are foreground.

Dilation

Expands objects by adding boundary pixels:

$(I \oplus S)[i, j] = \max\{I[i+u, j+v] : (u,v) \in S\}$

Output pixel is foreground if any pixel under structuring element is foreground.

Opening and Closing

Opening (erosion then dilation): removes small objects and smooths boundaries

$I \circ S = (I \ominus S) \oplus S$

Closing (dilation then erosion): fills small holes and connects nearby objects

$I \bullet S = (I \oplus S) \ominus S$

Structuring Elements

Common shapes:

- Square: [[1,1,1], [1,1,1], [1,1,1]]
- Cross: [[0,1,0], [1,1,1], [0,1,0]]
- Disk: circular approximation at various radii

4.0 Results

4.1 Original Grayscale image

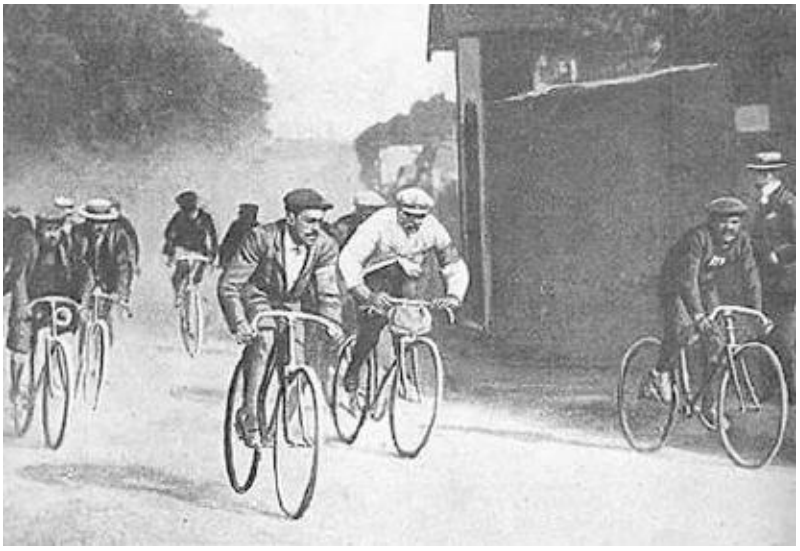


Figure 4.0-1: Grayscale Image

4.2 Original RGB Image

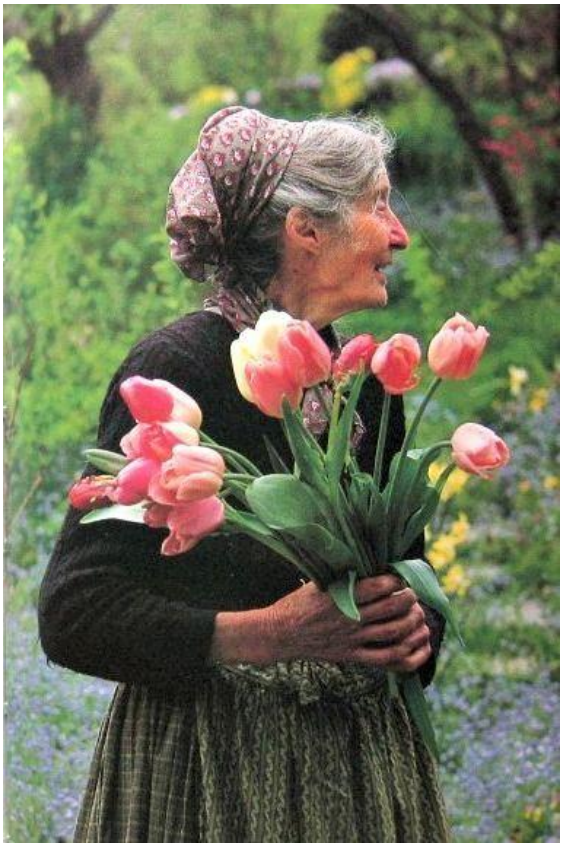


Figure 4.0-2: RGB Image

4.3 RGB to Grayscale Conversion

Figure 4.0-3: RGB to Grayscale Conversion

4.4 Median Filter

Figure 4.0-4: Median Filter

4.5 Mean Filter

Figure 4.0-5: Mean Filter

4.6 Gaussian Filter (size=5 , sigma=1)

Figure 4.0-6: Gaussian Filter

We notice that the median filter lowered the contrast of the image, it is mainly used to remove salt and peper noise. The mean filter and gaussian filter removed the noise by averaging. While the output of mean filter is much blurry than gaussian filter.

4.7 Gamma Corrected Image (gamma =2.2)

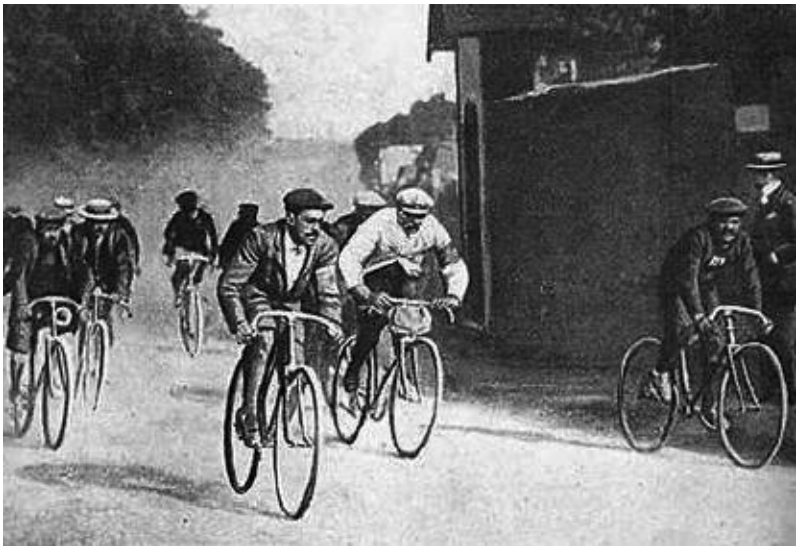


Figure 4.0-7: Gamma Corrected Image

4.8 Histogram Equalization

Figure 4.0-8: Histogram Equalization

We notice that the gamma corrected image is much smoother than the histogram equalized image.

4.9 Laplacian Filter

Figure 4.0-9: Laplacian Filter

4.10 Sobel Filter Magnitude

Figure 4.0-10: Sobel Filter Magnitude

4.11 Sobel Filter Direction

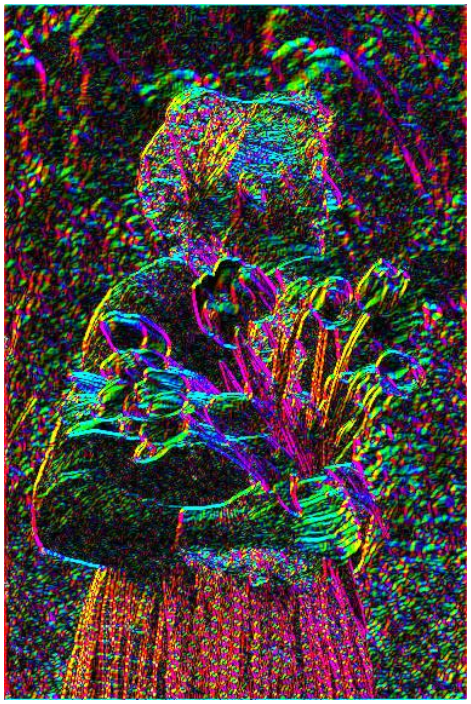


Figure 4.0-11: Sobel Filter Direction

We notice that the different colours indicate the edge direction, where red indicates 0° or 180°, yellow and green indicates 45°, cyan and light blue indicates 90°, blue and dark purple indicates 135°.

We also notice that Lapacian filter is much nosier than Sobel filter.

4.12 Global Thresholding (Threshold=120)

Figure 4.0-12: Global Thresholding

4.13 Otsu's Thresholding

Figure 4.0-13: Ostu's Thresholding

4.14 Adptive Thresholding (size =11 ,C=3)

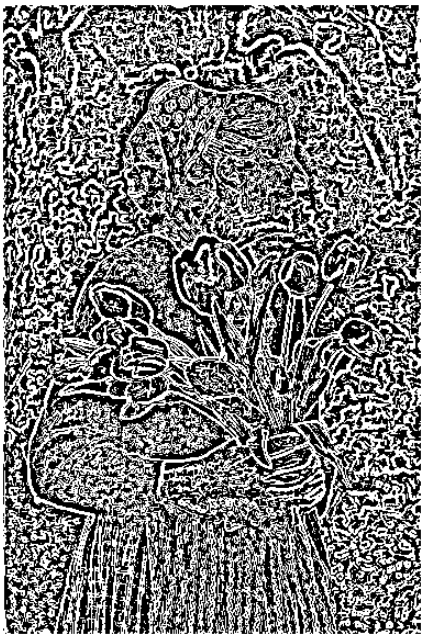


Figure 4.0-14: Adaptive Thresholding

We notice that the adaptive thresholding is noisy and needs much tunning. Global thresholding needs tunning as well. Otsu's thresholding outputs a great result in one shot.

4.15 Bit Slicing

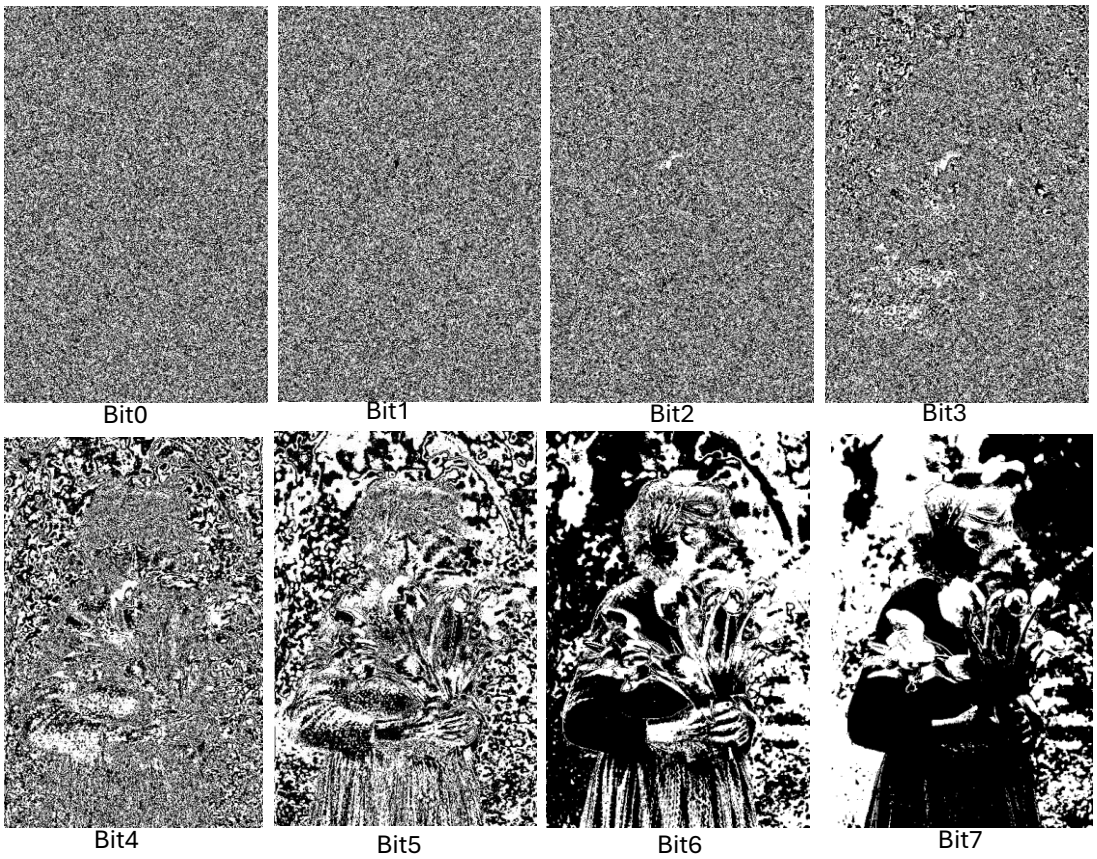


Figure 4.0-15: Bit Slicing

We notice that the LSB (bits 0-4) are noisy, while the MSB (bits 5-7) have the dominant features of the image

4.16 Adding Text

Figure 4.0-16: Adding Text on Grayscale Image

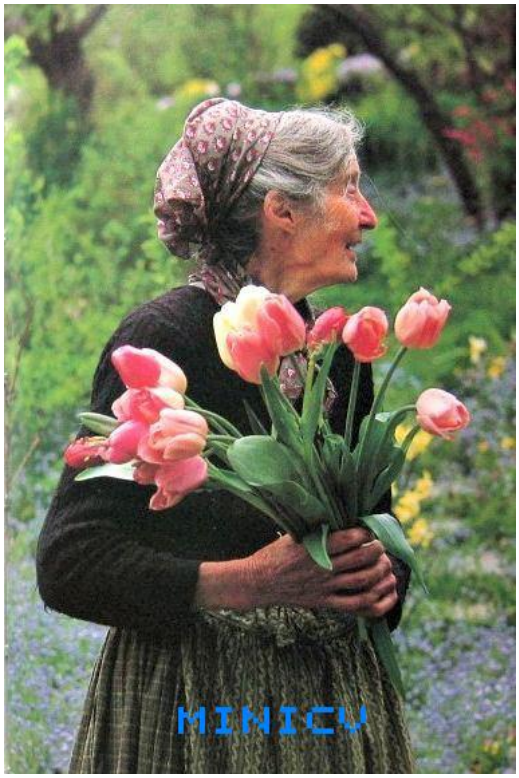


Figure 4.0-17: Adding Text on RGB Image

4.17 Adding Drawing

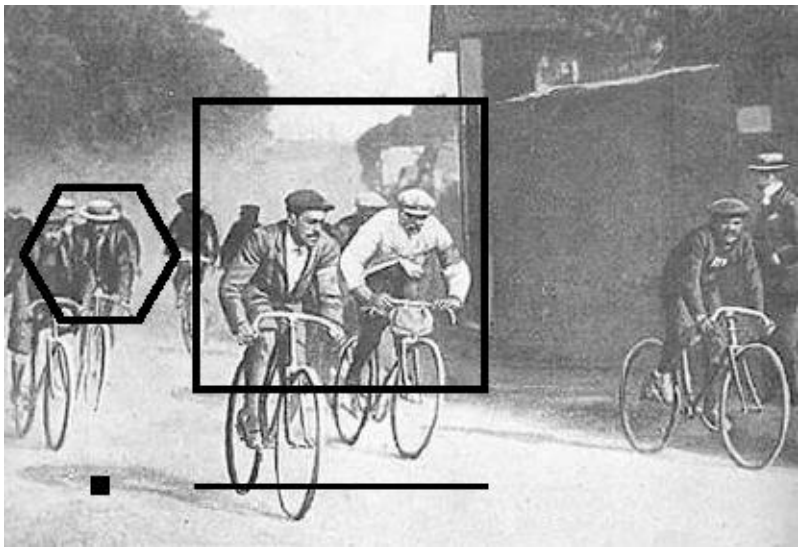


Figure 4.0-18: Adding Drawing on Grayscale Image

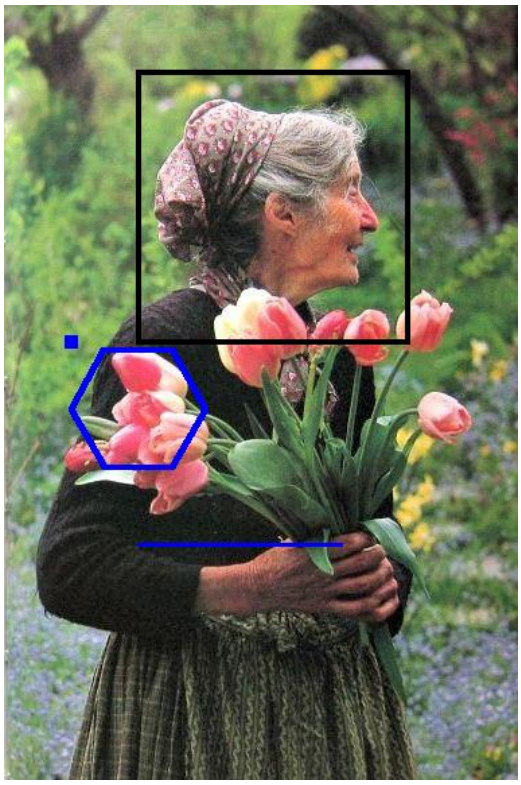


Figure 4.0-19: Adding Drawing on RGB Image

4.18 Core Operations (Image normalization, Pixel clipping, Padding)

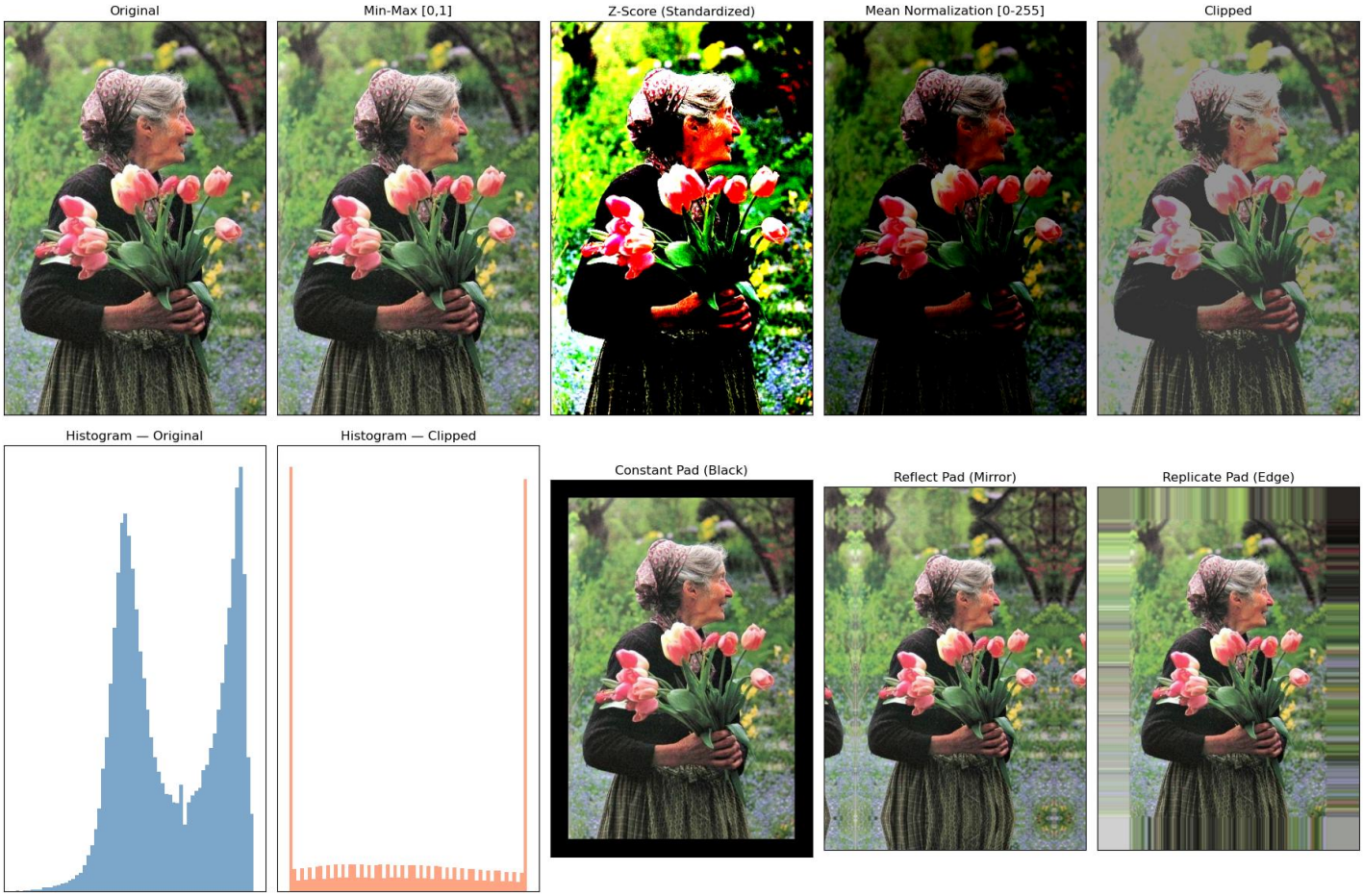


Figure 4.0-20 Normalize(min_max, z_score, mean_norm)- clipping(clipped image,original &clipped histogram)-Padding(constant, reflect, edge)

4.19 Feature Extractor

Figure 4.0-21Color Histogram, HOG Lite, Edge Histogram Descriptor

4.20 Geometric Transformations

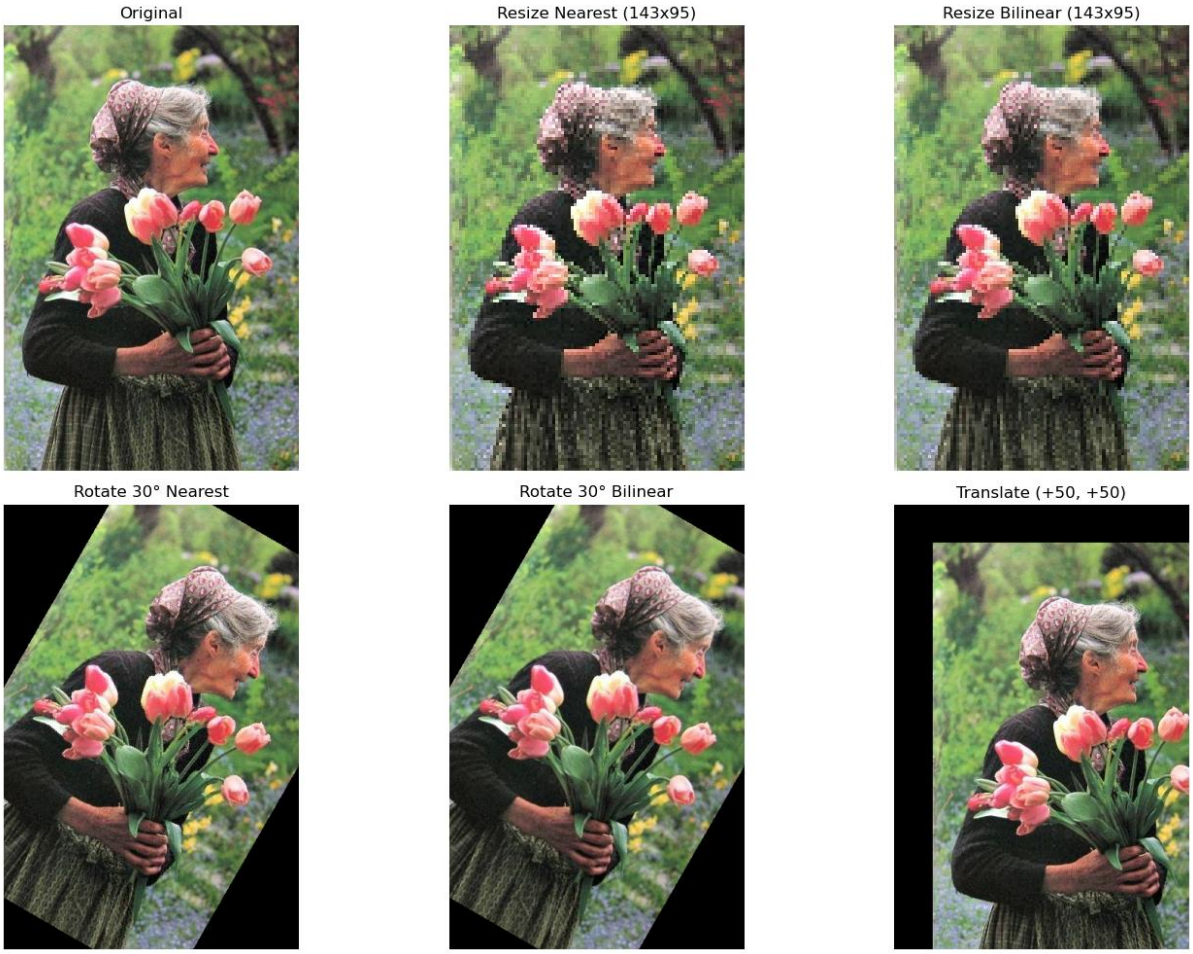


Figure 4.0-22 Resize (nearest-Bilinear)_Rotate(nearest-Bilinear)_Translate

5.0 Conclusion

Comprehensive mathematical foundations and algorithmic details were provided for all components of the MiniCV image processing library. Each technique has been explained with:

- Mathematical formulations and equations
- Algorithmic pseudocode and implementation strategies
- Computational considerations and optimization approaches
- Practical applications and use cases

The library implementation should follow these mathematical principles while adhering to software engineering best practices including modular design, comprehensive documentation, input validation, and performance optimization through NumPy vectorization.

6.0 Repository Link

https://github.com/Hla-3/MiniCV.git